soluções mobile

prof. Thyerri Mezzari

Android: Persistência de dados, Preferences e SQLite + Room Persistance Library

Persistência de Dados

Conceitualizando

Persistência de Dados

Além de toda a interatividade que um aplicativo Android Nativo deve fornecer ao usuário, em boa parte das situações alguns dados deverão "durar" além do uso atual do app. Uma lista de preferências do usuário, uma configuração a ser salva, registros do usuário logado no momento e etc. Uma hora ou outra você como desenvolvedor terá que garantir que "coisas" permaneçam "vivas" mesmo depois da aplicação estar "morta".

Persistência de Dados

O Android fornece várias opções para salvar os dados de aplicativos persistentes. A solução que for escolhida irá depender de necessidades específicas, como se os dados devem ser privados de sua aplicação ou acessíveis a outras aplicações (e de usuário) e quanto espaço os seus dados requerem.

Mas basicamente, as principais maneiras de persistir informações no Android com API nativa são: utilizando um par de chave valor, chamado de *SharedPreference*, ou usando um banco de dados relacional convencional, com o *SQLite*.

Persistência de Dados

Detalhamento de soluções clássicas

- **SharedPreferences**: Armazenar dados particulares primitivos em pares chave-valor.
- **SQLite Databases**: Armazenar dados estruturados em um banco de dados privado.

Outras soluções:

- **Firebase**: banco de dados noSQL na nuvem compatível com Android e iOS

Preferences

getPreferences x getSharedPreferences

SharedPreferences

A classe *SharedPreferences* permite salvar e recuperar pares de chave/valor de tipos de dados primitivos. Podemos usar o *SharedPreferences* para salvar os dados primitivos: *booleans*, *floats*, *ints*, *longs*, *e strings*. Estes dados irão persistir na sessão do usuário (mesmo que sua aplicação seja morta).

Este tipo de armazenamento seria o caminho ideal para salvar pequenos dados e configurações que não precisam ser sincronizadas na nuvem e pertence apenas a instalação atual do app. Por exemplo, os dados do usuário logado no momento em seu app, poderia ser salvos de alguma forma como Preference, bem como as preferências dele em relação ao "tema escuro ou claro" do aplicativo.

Escrevendo - *SharedPreferences*

O código abaixo escreve um número (***int*** = *número inteiro)* no banco de *SharedPreferences* cujo o nome é `com.satc.exemploapp.preferences.PERFIL`.

```kotlin
// kotlin
val sharedPref = activity?.getSharedPreferences("com.satc.exemploapp.preferences.PERFIL", Context.MODE_PRIVATE) ?: return
with (sharedPref.edit()) {
    putInt("IDADE_USUARIO", 33)
    commit()
}
```

Escrevendo - *SharedPreferences*

O código abaixo escreve um número (***int*** = *número inteiro)* no banco de *SharedPreferences*.
O nome da chave que dá acesso aos dados está armazenada em uma String do arquivo Strings.xml.

```
val sharedPref = activity?.getSharedPreferences(
        getString(R.string.preference_file_key), Context.MODE_PRIVATE)
```

```
with (sharedPref.edit()) {
    putInt(getString(R.string.saved_high_score_key), newHighScore)
    apply()
}
```

Escrevendo - *SharedPreferences*

Vale observar que para cada tipo de dado a ser salvo na memória, precisamos usar um método específico:

- **putInt**: salva um número inteiro
- **putFloat:** salva um número não-inteiro (com vírgula)
- **putString**: salva um text/string
- **putBoolean**: salva um valor true ou false

Lendo - *SharedPreferences*

Seguindo nosso exemplo de escrita anterior, caso precisássemos recuperar o valor da "idade do usuário" que salvamos como um int, poderíamos fazer assim:

```
val sharedPref = activity?.getPreferences(Context.MODE_PRIVATE) ?: return
val defaultValue = resources.getInteger(R.integer.saved_high_score_default_key)
val highScore = sharedPref.getInt(getString(R.string.saved_high_score_key), defaultValue)
```

*PS.: O segundo argumento das funções **getXXX** é o valor padrão caso ainda não tenha sido setado valor anteriormente.*

SQLite

SQLite - O banco de dados mobile

O **SQLite** é uma lib de funcionalidades escrita em C que também possui um banco de dados relacional embutido em sua estrutura.

SQLite não é um *client* usado para conectar com um grande servidor externo de banco de dados, mas sim o próprio servidor em si. A biblioteca SQLite lê e escreve diretamente no arquivo de banco de dados no disco.

O uso do **SQLite** é recomendado onde a simplicidade da administração, implementação e manutenção são mais importantes que incontáveis recursos que bancos de dados tradicionais, mais voltados para aplicações complexas, possivelmente implementam *(ex: Postgres ou Oracle)*.

SQLite - O banco de dados mobile

Por ser altamente portável, autocontido e extremamente leve, o **SQLite** se tornou a solução nativa de banco de dados para muitas plataformas móveis como o *iOS* e também o ***Android***.

Inicialmente o uso **SQLite** junto ao *Android* se dava de maneira bem clássica, trabalhando manualmente com uma série de pontos, inclusive a criação de tabelas no banco de dados. De certa forma é possível trabalhar desta maneira até os dias de hoje, principalmente se você estiver trabalhando com um app de pequeno a médio porte.

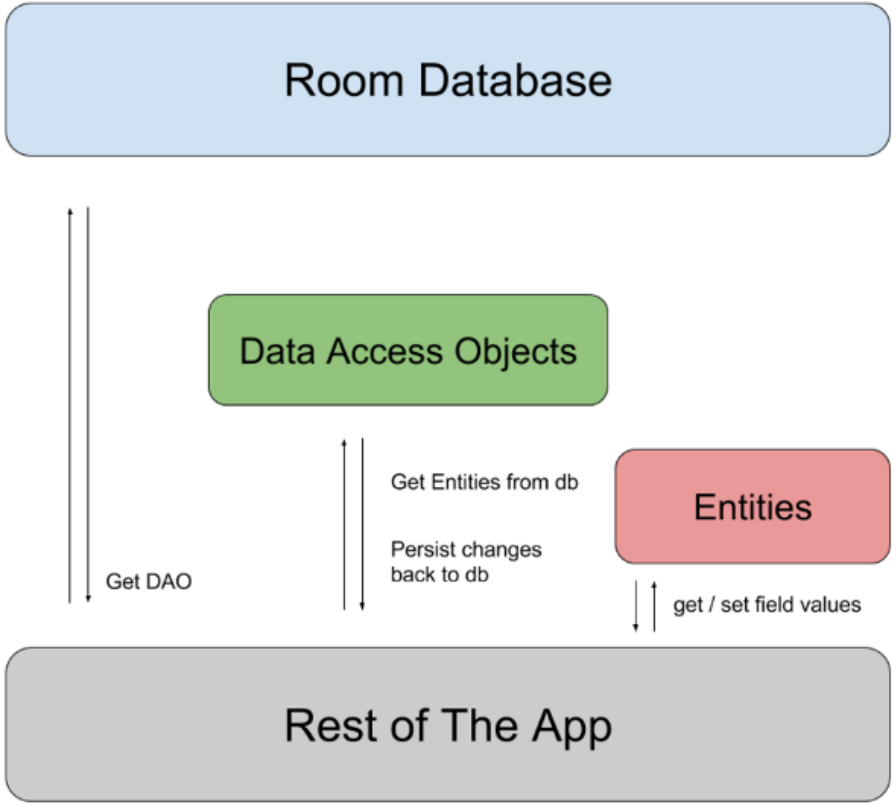
Porém em contraponto ao parágrafo anterior, atualmente com a evolução da plataforma uma nova maneira de lidar com o SQLite fora surgindo junto das últimas versões da Android SDK, **o banco de dados ROOM.**

https://developer.android.com/training/data-storage/sqlite?hl=pt-br

Room Persistence Library

RoomDatabase

Room Persistence Library



Room Persistence Library

A biblioteca de persistência **Room** oferece uma camada de abstração sobre o SQLite para permitir um acesso mais robusto ao banco de dados, aproveitando toda a capacidade do SQLite.

É altamente recomendado pelo Google usar o **Room** em vez do SQLite puro, mas sendo que o **ROOM** fora introduzido junto com as novas bibliotecas do *Android Jetpack* será necessário configurar seu projeto para que o mesmo rode.

Room Persistence Library

Para usar a **Room** no app, adicione as seguintes dependências ao arquivo *build.gradle* do app:

```
43  dependencies {
44
45      val room_version = "2.6.1"
46
47      implementation("androidx.room:room-runtime:$room_version")
48      annotationProcessor("androidx.room:room-compiler:$room_version")
49
50      ksp("androidx.room:room-compiler:$room_version")
```

Atenção com a compatibilidade de versões com o Kotlin, AGP, SDK e Android Studio. Se tiver problemas, use a room_version 2.6.1

https://developer.android.com/training/data-storage/room?hl=pt-br#setup

Room Persistence Library

Para usar a **Room** no app, acrescente o plugin ksp no gradle nível module:

```
1  plugins {
2      alias(libs.plugins.android.application)
3      alias(libs.plugins.kotlin.android)
4      alias(libs.plugins.kotlin.compose)
5      id("com.google.devtools.ksp") // plugin ksp
6  }
```

E também um novo plugin ksp no gradle nível de projeto:

```
2  plugins {
3      alias(libs.plugins.android.application) apply false
4      alias(libs.plugins.kotlin.android) apply false
5      alias(libs.plugins.kotlin.compose) apply false
6      id("com.google.devtools.ksp") version "1.9.10-1.0.13" apply false //ksp
7
8  }
```

Room Persistence Library

Lembre-se que sempre após alterar o gradle é preciso sincronizar novamente:

Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly. Sync Now

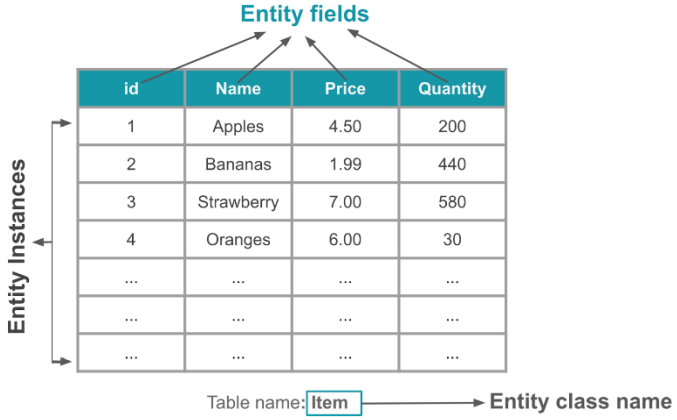
Entidade - Room Persistence Library

Para reproduzir a ideia de uma "tabela de contatos" com os campos name e email, iniciaremos nossa implementação usando **ROOM** criando o arquivo ***Entidade*** que irá representar cada linha de um "Contato" em nosso banco. Seria algo como um Modelo do modelo MVC:

```kotlin
// kotlin
@Entity(tableName = "contatos")
data class Contato(
    @PrimaryKey val id: Int,
    @ColumnInfo(name = "name") val name: String?,
    @ColumnInfo(name = "email") val email: String?
)
```

Classes de dados
As classes de dados são usadas principalmente para armazenar dados no Kotlin. Elas são definidas com a palavra-chave data.

Os objetos de classe de dados do Kotlin têm alguns benefícios extras. Por exemplo, o compilador gera utilitários automaticamente para comparar, mostrar e copiar, como toString(), copy() (link em inglês) e equals()



DAO - Room Persistence Library

Na sequência iremos criar a interface ***DAO*** responsável por definir quais métodos iremos utilizar para interação com a tabela usuarios. Coisas como todos os INSERTS previstos, possíveis tipos de SELECT com mais ou menos critérios, e até a funções para deletar usuários:

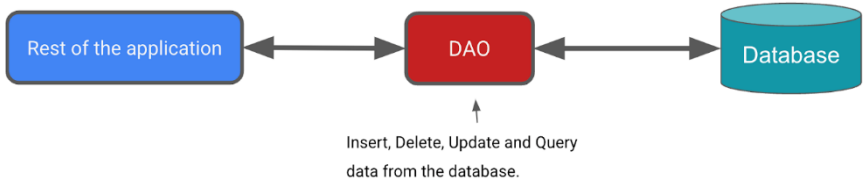
```
@Dao
interface ContatoDao {
  @Query("SELECT * FROM contatos WHERE id = :contatoId")
  fun getContato(contatoId: String): Contato

  @Query("SELECT * FROM contatos")
  fun getAll(): List<Contato>

  @Query("SELECT * FROM contatos WHERE name LIKE :name")
  fun loadAllByName(name: String): List<Contato>

  @Insert
  fun insertAll(vararg contatos: Contato)

  @Delete
  fun delete(contato: Contato)
}
```



RoomDatabase - Room Persistence Library

Por fim criaremos nossa classe que irá representar nosso banco de dados como um todo e também fornecer acesso rápido a nosso aplicativo a todas as instâncias do tipo DAO de cada tabela. O *companion object*

```
// kotlin - Database class com uma instância singleton.
@Database(entities = [Contato::class], version = 1, exportSchema = false)
abstract class ContatosDatabase : RoomDatabase() {

    abstract fun contatoDao(): ContatoDao

    companion object {
        @Volatile
        private var Instance: ContatosDatabase? = null

        fun getDatabase(context: Context): ContatosDatabase {
            // cria uma nova instancia do DB ou retorna se já tiver sido criada.
            return Instance ?: synchronized(this) {
               Room.databaseBuilder(context, ContatosDatabase::class.java, “contatos_database")
                    .build().also { Instance = it }
            }
        }
    }
}
```

Room - Acessando o banco

Sempre que precisarmos usar banco de dados em alguma *Activity* ou *Fragment* precisaremos usar a função estática ***getDatabase()*** que criamos para disparar o Singleton:

```
// kotlin

lifecycleScope.launch {
    val db = AppDatabase.getInstance(this@MainActivity)

    println(db.taskDao().getAll())
}
```

Room - Inserindo (INSERT) registros no banco

Para manipularmos os dados no DB, o mais correto nesse momento é acessar o banco por meio de um arquivo **Repository** e utilizar a classe **ViewModel** para manipular dados e eventos. Porém isso pode ser um pouco avançado neste momento.

Para facilitar e focar no conteúdo (banco de dados) podemos acessar a referência do nosso banco em uma *Activity* ou *Fragment*. Aqui teremos acesso aos **DAOs** disponíveis e os comandos escritos na interface.

Um destes comandos definidos por nós (desenvolvedores) foi o ***insertAll*** que serve para inserir novos contatos na tabela:

```
// kotlin
val novoContato:Contato = Contato(1, "Thyerri Mezzari",
"thyerri.mezzari@satc.edu.br") db.usuarioDao().insertAll(novoUsuario)
```

Room - Obtendo (SELECT) informações no banco

Para obtermos registros já salvos no banco, basta continuarmos usando nosso ***DAO*** e um dos métodos baseados em *SELECT*:

```
// kotlin
val contatoThyerri:Contato = db.contatoDao().getContato("1")
```

Na execução do método acima iríamos encontrar um usuário cujo o id (chave-primária) é "1".

Room - Outras necessidades

Podemos utilizar todos os métodos que nós mesmos tenhamos definido na interface DAO de cada tabela, UPDATE, DELETE e etc.

No mais recomendo dar uma olhada na documentação completa do **ROOM**:
https://developer.android.com/training/data-storage/room

E neste **tutorial** bem completo aqui:
https://developer.android.com/codelabs/basic-android-kotlin-compose-persisting-data-room?hl=pt-br#0

Android nativo - persistência

Exemplos de implementação

https://github.com/thyerrimezzari/BaseToDoListCompose

https://github.com/thyerrimezzari/ContatosAppComposeRoom

obrigado 🚀